Applied Intelligence (2024) 54:1893–1908
https://doi.org/10.1007/s10489-023-05129-8

KGTN-ens: few-shot image classification with knowledge graph ensembles

Dominik Filipiak[1,2] · Anna Fensel[3] · Agata Filipowska[4]

Accepted: 22 October 2023 / Published online: 25 January 2024
© The Author(s) 2024

Abstract
We propose KGTN-ens, a framework extending the recent Knowledge Graph Transfer Network (KGTN) in order to incorporate multiple knowledge graph embeddings at a small cost. There are many real-world scenarios in which the amount of data is severely limited (e.g. health industry, rare anomalies). Prior knowledge can be used to tackle this task. In KGTN, one can use a single knowledge source at once. The purpose of this study is to investigate the possibility of combining multiple knowledge sources. We evaluate it with different embeddings in a few-shot image classification task. Our model is partially trained on $k \in \{1, 2, 5, 10\}$ samples. We also construct a new knowledge source – Wikidata embeddings – and evaluate it with KGTN and KGTN-ens. With ResNet50, our approach outperforms KGTN in terms of the top-5 accuracy on the ImageNet-FS dataset for the majority of tested settings. For $k \in \{1, 2, 5, 10\}$ respectively, we obtained +0.63/+0.58/+0.43/+0.26 pp. (novel classes) and +0.26/+0.25/+0.32/–0.04 pp. (all classes).

Keywords Few-shot Image Classification · Knowledge Graph Enabled AI · Ensemble Learning

1 Introduction

Deep learning has made a substantial impact on a number of industrial and research areas. This includes computer vision, as the rapid development of representation learning started with the seminal work of [17] for the image classification task. However, numerous state-of-the-art models often require large amounts of data to train, which can be costly to gather and label – especially for vision-related tasks. Therefore, an intense research effort can be observed in the area of data-efficient machine learning methods. Few-shot learning (often abbreviated as FSL) is a machine learning task, where the machine learning model is (partially) trained on a small amount of data – part of the labelled data is available in standard amounts, whereas the other part consists of only a few (typically less than 10) samples per class. Few-shot learning can also suffer from selection bias since the decision boundaries need to be adjusted to a new few samples, which can contain irrelevant and misleading artefacts (such as a background colour). Hence the learning process is substantially more challenging.

One way to tackle the few-shot learning task is to use some prior knowledge of the labelled data. Knowledge Graph Transfer Network (KGTN), the recent work of [2], solves this problem by learning the prototypes from the external sources of knowledge and comparing them against extracted features from an input image. A similarity function scores the output of these two and yields the class probability distribution. These external sources of knowledge are represented as class correlation matrices. A vital element of this architecture is the knowledge graph transfer module (KGTM), which tries to learn class prototypes from knowledge graph embeddings using gated graph neural networks (GGTN) [20].

In the KGTN approach, one has to select a single knowledge source. Inspired by ensemble learning approaches, this

✉ Dominik Filipiak
dominik.filipiak@student.uibk.ac.at

Anna Fensel
anna.fensel@wur.nl

Agata Filipowska
agata.filipowska@ue.poznan.pl

1 University of Innsbruck, Innrain 52, Innsbruck 6020, Austria

2 University of Warsaw, Krakowskie Przedmieście 26/28, Warsaw 00-927, Poland

3 Wageningen University & Research, Droevendaalsesteeg 2, Wageningen 6708 PB, The Netherlands

4 Poznań University of Economics and Business, Al. Niepodległości 10, Poznań 61-875, Poland

observation leads us to the following questions: is it possible to learn prototypes from multiple knowledge graph embeddings? If so, will it result in higher performance metrics values, such as accuracy for classification problems? Therefore, we propose KGTN-ens, an extension of KGTN, that uses multiple embeddings instead of a single one. Each of them generates different prototypes, which are later combined and compared against the output of the feature extractor. We test two ensemble learning techniques in this paper. We also evaluated different combinations of three knowledge graphs, one of which (based on Wikidata) was introduced by us and has not been used in the original paper. Our solution is knowledge graph agnostic, provided that the knowledge graph is embedded and linked to the classes used in the image classification.

The contribution of this paper is two-fold: (1) we propose KGTN-ens, a new method based on KGTN, and evaluate it with different combinations of embeddings, (2) we construct a new knowledge source – Wikidata embeddings – and evaluate it with KGTN and KGTN-ens. We evaluate KGTN-ens with different combinations of embeddings in a few-shot image classification task. For a standard few-shot benchmark setup (ImageNet-FS dataset, ResNet-50 as a feature extractor), our approach outperforms KGTN in terms of the top-5 accuracy on the ImageNet-FS dataset for the majority of tested settings. Specifically, we achieved +0.63/+0.58/+0.43/+0.26 pp. (novel classes) and +0.26/+0.25/+0.32/–0.04 pp. (all classes) for $k \in \{1, 2, 5, 10\}$ respectively (averaged over 5 different runs). The method also extends the original KGTN approach by using not one, but multiple knowledge sources at a small computational cost. We also construct a new knowledge source – Wikidata embeddings – and evaluate it with KGTN and KGTN-ens. These embeddings may serve as a new knowledge source for other tasks beyond this study. The code available on GitHub[1]

The remainder of this paper is organised as follows. A comprehensive literature survey on related work is presented in Section 2. Section 3 provides a description of the KGTN-ens architecture. Section 4 describes the results of the evaluation of Wikidata embeddings with KGTN and KGTN-ens with different combinations of embeddings, along with the detailed analysis and ablation studies. Section 5 concludes the paper.

2 Related work

This section provides a comprehensive overview of the related work. We start with a brief review of the techniques used for graph neural networks, which are at the core of the KGTN-ens architecture. Then, we provide a short survey on recent advancements in few-shot learning, which is the main machine learning task solved by the architecture presented in this paper.

Graph neural networks In general, Graph neural networks (GNNs) represent a type of neural network, which processes the specified attributes of the graphs. Task tackled by GNN can be either node-level (such as prediction of a property for each node), edge-level (prediction of a property for each edge), or graph-level (prediction of a property for a whole graph) [28]. Following [16], a crucial feature of GNNs is being either invariant or equivariant to permutations. That is, for a graph $\mathcal{G}$, network f and a permutation Π we have $f(\Pi \star \mathcal{G}) = f(\mathcal{G})$ and $f(\Pi \star \mathcal{G}) = \Pi \star f(\mathcal{G})$ for invariance and equivariance respectively. The general-purpose models from the state-of-the-art family of transformer architectures [32] can be viewed as a special instance of a graph neural network. Graph neural networks have a wide area of applications, with notable examples in biology (e.g. protein interface prediction) or social networks (e.g. community detection or link prediction). The less obvious application of GNNs is in the field of image classification, where they are used to learn the prototypes from the knowledge graph embeddings in a few-shot learning setting.

GNNs fall into a broader category of geometric deep learning, which is devoted to the application of deep neural networks on structured non-Euclidean domains, such as graphs, manifolds, meshes, or grids [1]. Gilmer et al. [13] proposed *message passing*, which is one of the most important concepts in GNNs. In this approach, nodes and/or edges can rely on their neighbours in order to create meaningful embeddings iteratively. Wu et al. [37] classify GNNs into four broad categories: recurrent GNNs (RecGNN), convolutional GNNs (ConvGNNs), graph autoencoders (GAEs), and spatial-temporal GNNs (STGNNs). In this article, Gated Graph Neural Network (GGNN) [20] are of special interest. They belong to the category of RecGNNs. For a fair comparison with KGTN (our baseline), we used GGNN in our experiments. Following [20], the intuitive difference between GNN and GGNN relies on the explicit graph structure of GNNs, which results in more generalisation capabilities at the expense of a less general model of the latter.

Few-shot learning While being very effective for numerous vision tasks, one of the main problems with convolutional neural networks (or machine learning in general) is the amount of data they need to provide meaningful predictions. More recent architectures, such as self-attention models require even more data to train. On the contrary, humans typically require only a few samples to acquire knowledge of seen objects. One way to tackle this issue is few-shot learning, which is aimed at learning from scarce data. The complexity of the problem often stems from the required sudden shift

[1] https://github.com/DominikFilipiak/KGTN-ens

of decision boundaries, which is hard to achieve using only a few samples. A special case of few-shot learning is one-shot learning, which is learning from one labelled sample per class.

Following [31], few-shot learning methods can be divided into data augmentation, transfer learning, meta-learning, and multimodal learning. Data augmentation techniques aim to artificially extend the amount of available data by either transforming input data [3] or resulting features [4]. Transfer learning focuses on resuing features from networks trained on different datasets with the required amount of data by techniques such as pre-training and fine-tuning or domain adaptation. Meta-learning includes techniques devoted to learning from data and tasks in order to reuse this knowledge for future downstream tasks. Finn et al. [10] proposed a model agnostic meta-learning algorithm MAML. Specialised approaches to meta-learning include neural architecture search [8] or metric learning [5, 11]. Finally, multimodal learning focuses on the incorporation of external knowledge from heterogenous domains, such as text, speech or knowledge graphs [35].

The concept of prototypes was introduced in the work of [30], where they proposed prototypical networks focused on learning metric space between class instances and their prototypes. Hariharan and Girshick [14] used representation regularisation and introduced the concept of *hallucinations* in order to enlarge the number of available representations during the training. Wang et al. [36] employed meta-learning techniques and combined them with the aforementioned *hallucinations* to improve few-shot classification metrics. A growing number of scholars incorporate structured knowledge into their computer vision research [23]. For instance, [19] studied transferable features with the hierarchy which encodes the semantic relations. Their approach turned out to apply to the problem of zero-shot learning as well. Shen et al. [29] proposed model agnostic regularisation technique in order to leverage the relationship between graph labels to preserve category neighbourhood.

3 Method

This section explains the details of KGTN-ens. The method extends the KGTN architecture proposed by [2], which relies on graph-based knowledge transfer to yield state-of-the-art results on few-show image classification. The most important difference relies on the usage of multiple graphs instead of a single one, which enables the usage of different knowledge sources. Each of these graphs generates different prototypes, which are later combined and compared against the output of the feature extractor. It might be not immediately obvious why the approach with multiple knowledge graphs is used, as they may be merged into one using `owl:sameAs` or similar property. Notice that this method does not require knowledge graphs in a strict sense – KGTM processes only distances between classes, which are later used for scoring prototypes. Therefore, integrating different sources of knowledge is fairly easy and requires a minimum amount of effort – the KGTN-ens architecture seamlessly handles different types of distances derived from embeddings.

Problem formulation Following [2], the classification task is formulated as learning the prototypes of considered classes. In the typical approach to classification, the model prediction $\hat{y}$ based on the input x is obtained in the following way:

$$\hat{y} = \underset{k}{\operatorname{argmax}}\ p(y = k \mid x) \tag{1}$$

where p is calculated using the standard softmax function:

$$p(y = k \mid x) = \frac{\exp\left(f_k(\mathbf{x})\right)}{\sum_{i=1}^{K} \exp\left(f_i(\mathbf{x})\right)}, \tag{2}$$

where K is the number of considered classes and f_k is the linear classifier. Since

$$\underset{k}{\operatorname{argmax}}\ p(y = k \mid \mathbf{x}) = \underset{k}{\operatorname{argmax}}\ f_k(\mathbf{x}), \tag{3}$$

the $f_k(\mathbf{x})$ can be formulated as follows:

$$f_k(\mathbf{x}) = \mathbf{w}_k^T \mathbf{x} + b_k = -\frac{1}{2}\|\mathbf{w}_k - \mathbf{x}\|_2^2 + \frac{1}{2}\|\mathbf{w}_k\|_2^2 + \frac{1}{2}\|\mathbf{x}\|_2^2 + b_k$$

setting $b_k = 0$ and $\|\mathbf{w}_i\|_2 = \|\mathbf{w}_j\|_2$ for each i, j, the classifier $f_k\mathbf{x}$ can be perceived as a similarity measure between the extracted features and prototypes:

$$\hat{y} = \underset{k}{\operatorname{argmax}}\ p(y = k \mid \mathbf{x}) = \underset{k}{\operatorname{argmax}}\ \|\mathbf{w}_k - \mathbf{x}\|_2^2\,. \tag{4}$$

As a result, $\mathbf{w}_k$ can be interpreted as a prototype for class k, and these prototypes are learned during the training process.

The overall architecture of KGTN-ens is presented in Fig. 1 and it consists of three main parts: *Feature Extractor*, *KGTMs*, and *Prediction with ensembling*. Feature Extractor is a convolutional neural network that extracts features from the input image, such as ResNet [15]. KGTMs refer to the list of knowledge graph transfer modules (each one handles a different knowledge graph) that are used to generate prototypes. Finally, prediction with ensembling a module that scores extracted features against obtained prototypes in order to make the final classification.

KGTMs Since we use the plain ResNet50 for the feature extractor part, we start the description with the KGTMs part. Consider a dataset of images, where each of them is

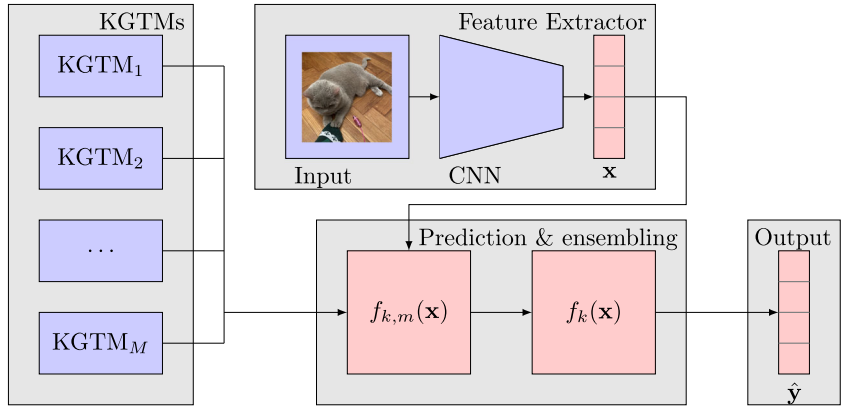


Fig. 1 Architecture of KGTN-ens

associated with either a base class or a novel class. There are $K_{\texttt{base}}$ base classes and $K_{\texttt{novel}}$ novel classes ($K = K_{\texttt{base}} + K_{\texttt{novel}}$). In the original KGTN approach, the correlations between categories are encoded in a graph $\mathcal{G} = \{\mathbf{V}, \mathbf{A}\}$, where $\mathbf{V} = \{v_1, v_2, \ldots, v_{K_{\texttt{base}}} \ldots, v_K\}$ represents classes and $\mathbf{A}$ denotes an adjacency matrix, in which $A_{i,j}$ is the *correlation* between classes v_i and v_j. Our approach extends this concept in a way in which there are multiple graphs $\mathcal{G}_1, \ldots, \mathcal{G}_M$. Specifically, each of them shares the same classes $\mathbf{V}$ but has different correlation values stored in $\mathbf{A}$ matrices.

Just as KGTN, KGTN-ens is based on Gated Graph Neural Network [20], in which each class is represented by a node v_k is associated with a hidden state h_k^t at time t. It is initialised with $\mathbf{h}_k^0 = \mathbf{w}_k^{\text{init}}$, where $\mathbf{w}_k^{\text{init}}$ are chosen at random. The parameter vector $\mathbf{a}_k^t$ for node k at time $t \in \{1, \ldots, T\}$ is defined as:

$$\mathbf{a}_k^t = \left[\sum_{k'=1}^{K} a_{kk'} \mathbf{h}_{k'}^{t-1}, \sum_{k'=1}^{K} a_{k'k} \mathbf{h}_{k'}^{t-1} \right], \tag{5}$$

where $a_{kk'}$ denotes the correlation between nodes k and k'. The hidden states $\mathbf{h}_k^t$ for weight k at time t are determined with a gating mechanism inspired by GRU (abbr. from the gated recurrent unit), which was introduced by [6]:

$$\begin{aligned} \mathbf{z}_k^t &= \sigma(\mathbf{W}^z \mathbf{a}_k^t + \mathbf{U}^z \mathbf{h}_k^{t-1}), \\ \mathbf{r}_k^t &= \sigma(\mathbf{W}^r \mathbf{a}_k^t + \mathbf{U}^r \mathbf{h}_k^{t-1}), \\ \widetilde{\mathbf{h}}_k^t &= \tanh\left(\mathbf{W}\mathbf{a}_k^t + \mathbf{U}(\mathbf{r}_k^t \odot \mathbf{h}_k^{t-1})\right), \\ \mathbf{h}_k^t &= (1 - \mathbf{z}_k^t) \odot \mathbf{h}_k^{t-1} + \mathbf{z}_k^t \odot \widetilde{\mathbf{h}}_k^t. \end{aligned} \tag{6}$$

Here, $\mathbf{W}^z$ and $\mathbf{U}^z$ are the weights for the update gate, and $\mathbf{W}^r$ and $\mathbf{U}^r$ are the weights for the reset gate. The hyperbolic tangent function is given by tanh, whereas σ is the sigmoid function. The final weight $\mathbf{w}_k^*$ for class k is defined as:

$$\mathbf{w}_k^* = o(\mathbf{h}_k^T, \mathbf{h}_k^0), \tag{7}$$

where o is the fully connected layer.

Prediction and ensembling The classifier $f(\mathbf{x})$ is treated as a similarity metric between the output of the feature extractor and the most similar class prototypes learned by the knowledge graph transfer module. In the original KGTN approach, the relationship between these two was calculated using the inner product, cosine similarity or Person's correlation coefficient. For the inner product, which was the most effective, the classifier was defined as

$$f_k(\mathbf{x}) = \mathbf{x} \cdot \mathbf{w}_k^*, \tag{8}$$

where x is the feature vector of an image and $\mathbf{w}_k^*$ denotes the learned weight for class k. Cosine similarity is defined as

$$f_k(\mathbf{x}) = \left(\mathbf{x} \cdot \mathbf{w}_k^*\right) \cdot \left(\|\mathbf{x}\|_2 \left\|\mathbf{w}_k^*\right\|_2\right)^{-1}, \tag{9}$$

whereas Person's correlation coefficient is given by

$$f_k(\mathbf{x}) = \left((\mathbf{x} - \bar{\mathbf{x}}) \cdot (\mathbf{w}_k^* - \bar{\mathbf{w}}_k^*)\right) \left((\|\mathbf{x} - \bar{\mathbf{x}}\|_2) \left(\left\|\mathbf{w}_k^* - \mathbf{w}_k^*\right\|_2\right)\right)^{-1}. \tag{10}$$

Here, $\bar{\mathbf{x}}$ and $\bar{\mathbf{w}}_k^*$ are respectively the mean values of $\mathbf{x}$ and $\mathbf{w}_k^*$ repeated to match the shape of $\mathbf{x}$ and $\mathbf{w}_k^*$. Conventionally,

$$f(\mathbf{x}) = \arg\max_k f_k(\mathbf{x}). \tag{11}$$

However, in our approach, we use the ensembling-inspired technique to improve the performance of the classifier.

In KGTN-ens, we calculate similarity for each of the m available graphs. Using a similar inner product approach, this is done the following way: $f_{k,m}(\mathbf{x}) = \mathbf{x} \cdot \mathbf{w}_{k,m}^*$, where $\mathbf{w}_{k,m}^*$ is the learned weight for the m-th graph. Then, the final result for class k has to be chosen. Such an approach is inspired by ensemble learning strategies, though we do not use *weak learners* in a strict sense. One of the main drawbacks of ensemble learning – the linear memory complexity with the proportional computational burden – is partially avoided,

as only the part of the network is multiplied. Most importantly, the feature extractor, which often can be the largest component of modern architectures, is used only once. This enables us to fit several knowledge sources on proprietary GPUs (we used a single NVIDIA RTX 2080 Ti in our experiments). We propose two simple approaches for selecting the final result: mean and maximum. For the former, the result for class k is the mean of the m products:

$$f_k(\mathbf{x}) = \frac{1}{M} \sum_{m=1}^{M} f_{k,m}(\mathbf{x}). \tag{12}$$

In ensemble learning literature, this would be called *soft voting*. The maximum approach is very similar:

$$f_k(\mathbf{x}) = \max_{m=1}^{M} \left(f_{k,m}(\mathbf{x}) \right), \tag{13}$$

In other words, we take the maximum of the similarities for each of the m available graphs.

Optimisation To enable fair comparison, we use a two-step training regime similar to [14] and [2] – the first is devoted to the feature extractor, whereas the second one fine-tunes the graph-related part of the network. In the first stage, we train the feature extractor $\phi(\cdot)$ using the base classes from $\mathcal{D}_{base}$. The loss $\mathcal{L}_1$ calculated in this step consists of the standard cross-entropy loss and squared gradient magnitude loss [14], which acts as a regularisation term:

$$\mathcal{L}_1 = \mathcal{L}_c + \lambda \mathcal{L}_s, \tag{14}$$

where:

$$\mathcal{L}_c = -\frac{1}{N_{\text{base}}} \sum_{i=1}^{N_{\text{base}}} \sum_{i=1}^{K_{\text{base}}} \mathbb{1}_{k=y_i} \log p_i^k, \tag{15}$$

$$\mathcal{L}_s = \frac{1}{N_{\text{base}}} \sum_{i=1}^{N_{\text{base}}} \sum_{i=1}^{K_{\text{base}}} \left(p_i^k - \mathbb{1}_{k=y_i} \right) \|\mathbf{x}_i\|_2^2, \tag{16}$$

where $\mathbb{1}$ is the indicator function and λ is a loss balance parameter. In the second stage, the weights of the feature extractor are frozen. Other parts of the architecture are trained using base and novel samples with the following loss:

$$\mathcal{L}_2 = -\frac{1}{N} \sum_{i=1}^{N} \sum_{i=1}^{K} \mathbb{1}_{k=y_i} \log p_i^k + \eta \sum_{k=1}^{K} \left\| \mathbf{w}_k^* \right\|_2^2, \tag{17}$$

where η balances the loss components.

4 Evaluation

This section contains the results of the conducted experiments. First, we introduce the used knowledge sources – semantic similarity graph, WordNet and Wikidata. Then, we describe the evaluation of KGTN-ens with different combinations of embeddings and compare them with the previous work. Finally, we provide a detailed analysis and ablation studies.

4.1 Knowledge sources

In our evaluation, we use three different sources of knowledge, which can be the backbone of KGTMs: *hierarchy*, *glove*, and *wiki*. The first two have been proposed by [2]. The *wiki* graph is constructed on top of Wikidata, a collaborative knowledge graph connected to Wikipedia [34]. In this subsection, we discuss the preparation of these knowledge sources in detail.

Semantic similarity graph (*glove*) The first source of knowledge is built from GLoVe word embeddings [26]. For two words w_i and w_j, their semantic distance $d_{i,j}$ is defined as the Euclidean distance between their GLoVe embeddings $\mathbf{f}_i^w$ and $\mathbf{f}_j^w$. Following [2], the final correlation coefficient $a_{i,j}$ is obtained using the following function:

$$a_{i,j} = \lambda^{d_{i,j} - \min\{d_{i,k} | k \neq i\})}, \tag{18}$$

where $\lambda = 0.4$ and $a_{i,i} = 1$.

WordNet category distance (*hierarchy*) This source of knowledge is built from the WordNet hierarchy – a popular lexical database of English [22]. Since ImageNet classes are based on WordNet, the WordNet hierarchy can be used to measure the distance between two classes. This time the distance $d_{i,j}$ is defined as the number of common ancestors of the two words (categories) w_i and w_j. The output is processed similarly to (18), except that the λ parameter is set to 0.5.

Wikidata embeddings (*wiki*) The last source of knowledge is built from the Wikidata embeddings. The mapping between the ImageNet classes and Wikidata is provided by [9]. Having the mapping, the class-corresponding entities from Wikidata can be embedded and used as class prototypes. Although there exist some datasets of Wikidata embeddings, they are often incomplete. Most importantly, they do not contain all the embeddings of ImageNet classes. Wembedder [24] offers 100-dimensional Wikidata embeddings made using the word2vec algorithm [21], but it is based on an incomplete dump of Wikidata and does not contain all the classes needed in the ImageNet-FS dataset. Zhu et al. [38] proposed Graphvite, a general graph embedding engine. *Wikidata5m* is a large dataset of 5 million Wikidata entities, which is used

to train the embeddings. The framework comes with embeddings created using numerous popular algorithms, such as TransE, DistMult, ComplEx, SimplE, RotatE, and QuatE. However, 891 out of 1000 entities used in the ImageNet are embedded, which was not enough for performing the experiment.

We used the pre-trained 200-dimensional embeddings of Wikidata entities from PyTorch BigGraph [18], which are publicly available[2]. The embeddings were prepared using the full Wikidata dump from 2019-03-06. All but three entities were directly mapped to embeddings to their Wikidata ID. Three entities (`Q1295201`, `Q98957255`, `Q89579852`) could not be instantly matched – they were manually matched to `"grocery store"@en`, `"cricket"@en`, and `Q655301` respectively. Having the mapping, now we create an embedding array, ordered as the mappings in the original KGTN paper (that is, as a 1000×200 array, where 200 denotes the dimensionality of a single embedding). The same function from (18) was used to generate final correlations between the embeddings, although this time $\lambda = 0.32$ was used (see Section 4.3).

4.2 Experiment results

In this subsection, we present the results of the conducted experiments. We describe the evaluation data – the experiment has been conducted on the ImageNet-FS dataset. The training hyperparameters and the setup are also described. We also describe the evaluation protocol, as well as the evaluation metrics. Finally, we present the results of the experiments and compare them with the previous work.

Data Similarly to Chen et al., our approach has been evaluated on ImageNet-FS, a popular benchmark for the few-shot learning task. ImageNet-FS contains 1,000 classes from ImageNet Large Scale Visual Recognition Challenge 2012 [27], of which 389 belong to the base category and 611 to the novel category. 193 base classes and 300 novel classes are used for training and cross-validation, whereas the test phase is performed on the remaining 196 categories and 311 novel classes. Base categories consist of around 1280 train and 50 test images per class. The authors of KGTN also evaluated their solution against a larger dataset, ImageNet-6K, which contains 6,000 classes (of which 1,00 belong to the novel category). Unfortunately, we were unable to test KGTN-ens using this dataset, since it has not been made public nor available to us at the time of writing this paper.

Training To enable fair comparison, we used the same 2-step training and evaluation procedures as in KGTN [2], which was drawn on SGM [14]. Stochastic gradient descent (SGD) was used to train the model with a batch size equal to 256 (divided equally for base and novel classes), a momentum of 0.9, and a weight decay of 0.0005. The learning rate is initially set at 0.1 and divided by 30 at every 30 epochs. In general, we used the same feature extractor and hyperparameters as in KGTN unless stated otherwise. Specifically, we use ResNet-50 – same as in e.g. KGTN and SGM. Using the terminology from the ResNet paper [15], we use features from the output of the last convolutional block in the last *stage*. For ResNet-50, that's `conv5_3`, as the 4th stage consists of the three blocks.

Setup All the experiments have been conducted on a single NVIDIA GeForce RTX 2080 Ti GPU. We used the code released by the authors of KGTN and modified it to support the KGTN-ens approach. PyTorch [25] was used to conduct the experiments.

Evaluation Following previous work in few-shot learning, we report our evaluation results in terms of the top-5 accuracy of novel and all (base + novel) classes in the k-shot learning task, where $k \in \{1, 2, 5, 10\}$ is the number of available training samples belonging to the novel category. Following [14] and [2], we repeat each experiment five times and report the averaged values of the top-5 accuracy. Table 2 shows the classification results compared with some of the recent state-of-the-art benchmarks. Figure 2 presents the top-5 accuracy of the KGTN-ens model on ImageNet-FS. Of three possible combinations of the three sources of knowledge, the KGTN-ens model performed best with the combination of *hierarchy* and *glove*. Notably, it performed better than

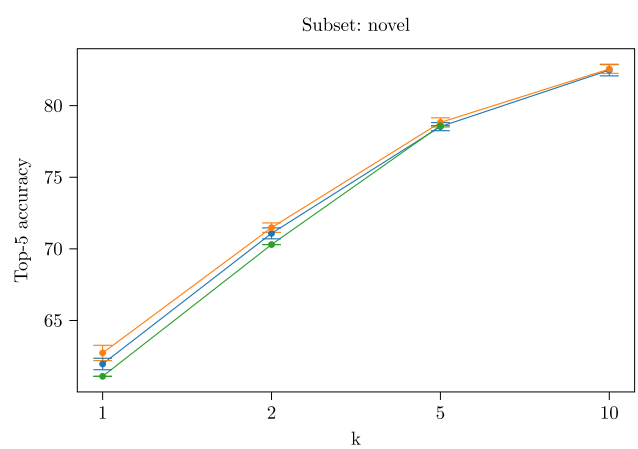
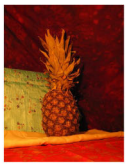


Fig. 2 KGTN-ens (blue, 5 runs averaged) performance mean top-5 accuracy compared to the KGTN (orange) and SGM with graph regularisation [29] (green). KGNT-ens uses *glove* and *hierarchy* graphs combined with the max ensembling function. Horizontal lines indicate standard deviations (not available for SGM with graph regularisation)

[2] https://torchbiggraph.readthedocs.io/en/latest/pretrained_embeddings.html

Table 1 Qualitative results (only novel) for the KGTN-ens models trained with ResNet50, max ensembling method and inner product similarity function

ImageNet samples predicted correctly in model(s) trained with...				
(none)	$k \in \{10\}$	$k \in \{5, 10\}$	$k \in \{2, 5, 10\}$	$k \in \{1, 2, 5, 10\}$
val/00041669	val/00047685	val/00048884	val/00044236	val/00000935
val/00032298	val/00015555	val/00038079	val/00024212 	val/00003270
val/00042517	val/00004622	val/00031416	val/00041116	val/00000366
val/00034003	val/00039617	val/00021424	val/00030855	val/00023021
val/00032298	val/00015555	val/00038079	val/00024212	val/00003270

The leftmost column contains the *hardest* images for KGTN-ens in terms of top-5 accuracy, as even for $k = 10$ the model was not able to predict it correctly
Images source: ImageNet (ILSVRC2012 validation set)

KGTN with these two sources of knowledge alone. Compared to KGTN (with inner product similarity and *glove* embeddings), the KGTN-ens model (inner product, max ens. function, *glove* and *hierarchy* embeddings) achieved +0.63, +0.58, +0.43, +0.26 pp. top-5 accuracy on novel classes for $k \in \{1, 2, 5, 10\}$ respectively. The smaller the k, the higher the performance gain. It also beats the more recent graph-based framework proposed by [29] by +1.73/+1.18/+0.20 pp. top-5 accuracy on novel classes. For all classes, the KGTN-ens model achieved +0.26, +0.25, +0.32, –0.04 pp. top-5 accuracy compared to the same KGTN model for $k \in \{1, 2, 5, 10\}$ respectively. Qualitative results for the best model are presented in Tables 1, and 2.

4.3 Details, ablation studies, and discussion

This subsection provides more details on the KGTN-ens model and its ablation studies. We analyse the impact of the following factors on the performance of the KGTN-ens model: adjacency matrices, used embeddings, ensembling method, similarity function, and variance of the results (Table 3).

Adjacency matrix analysis Since *glove* knowledge graph was the most effective for KGTN, we assume that *wiki* should roughly resemble it in terms of its distribution. In order to investigate the similarity between distributions, adjacency matrices have been created using pairwise Euclidean distances. While *glove* and *wiki* are normal-like, the distribution

Table 2 Top-5 accuracy on *novel* and *all* subsets on ImageNet-FS. All the methods used ResNet-50 as a feature extractor

	Novel				All			
	1	2	5	10	1	2	5	10
Baseline (only ResNet-50)	28.2	51.0	71.0	78.4	54.1	67.7	79.1	83.2
MN [33]	53.5	63.5	72.7	77.4	64.9	71.0	77.0	80.2
PN [30]	49.6	64.0	74.4	78.1	61.4	71.4	78.0	80.0
SGM [14]	54.3	67.0	77.4	81.9	60.7	71.6	80.2	83.6
SGM w/ G [14]	52.9	64.9	77.3	82.0	63.9	71.9	80.2	83.6
AWG [12]	53.9	65.5	75.9	80.3	65.1	72.3	79.1	82.1
PMN [36]	53.3	65.2	75.9	80.1	64.8	72.1	78.8	81.7
PMN w/ G [36]	54.7	66.8	77.4	81.4	65.7	73.5	80.2	82.8
LSD [7]	57.7	66.9	73.8	77.6	–	–	–	–
KTCH [19]	58.1	67.3	77.6	81.8	–	–	–	–
IDeMe-Net [3]	60.1	69.6	77.4	80.2	–	–	–	–
KGTN-CosSim [2]	61.4	70.4	78.4	82.2	67.7	74.7	80.9	83.6
KGTN-PearsonCorr [2]	61.5	70.6	78.5	82.3	67.5	74.4	80.7	83.5
KGTN-InnerProduct [2]	62.1	70.9	78.4	82.3	68.3	75.2	80.8	83.5
SGM with graph regularisation [29]	61.1	70.3	78.6	–	–	–	–	–
KGTN-ens (ours)	62.73	71.48	78.83	82.56	68.58	75.45	81.12	83.46

Partially based on the data provided by [14] and [2]

for *hierarchy* is bimodal and most of the distances are the highest ones (Table 4, Figs. 3 and 4). To assess the correlation between adjacency matrices, Mantel tests have been performed. The values marked as processed were run through (18). Correlations of the processed matrices are visibly higher compared to raw ones, especially regarding *glove* and *wiki*. The highest correlation has been observed between *glove* and *wiki*.

Importance of used knowledge graphs Firstly, we analyse the influence of the used KGs separately (without ensembling) – that is, with the original KGTN architecture. Table 5 shows the results of the ablation studies on the three knowledge graphs. The *hierarchy* and *glove* knowledge graphs are the ones examined by [2], whereas the *wiki* knowledge graph is the one introduced in our experiments. In order to ensure that the advantage comes from the knowledge encoded in KGs, Chen et al. argue that *glove* and *hierarchy* embeddings perform better than uniform (all correlations set to $1/K$) and random (correlations drawn from the uniform distributions) distance matrices. Similarly, the usage of *wiki* knowledge graph yielded generally better results (up to +3.44 pp for 1-shot in the novel category) compared to random and uniform cases, which constitutes a noticeable improvement. However,

Table 3 Descriptive statistics about the adjacency matrices

	KG	Min	Avg	Max	Std
Raw	*hierarchy*	0.00	9.76	10.00	1.20
	glove	0.00	8.52	14.31	1.29
	wiki	0.00	5.82	12.73	1.32
Processed	*hierarchy*	0.00	0.01	2.00	0.07
	glove	0.00	0.05	2.00	0.11
	wiki	0.00	0.08	2.00	0.14

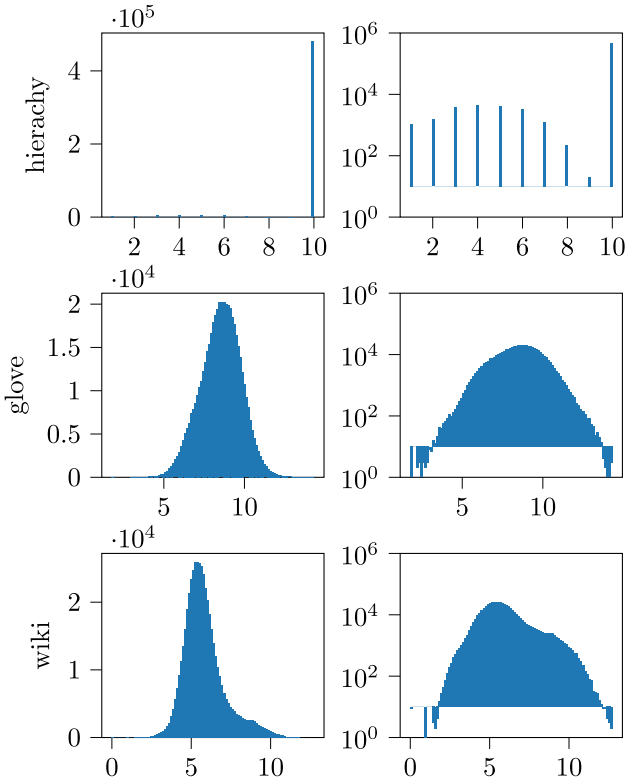


Fig. 3 Adjacency matrix distributions, raw values (normal and log scales)

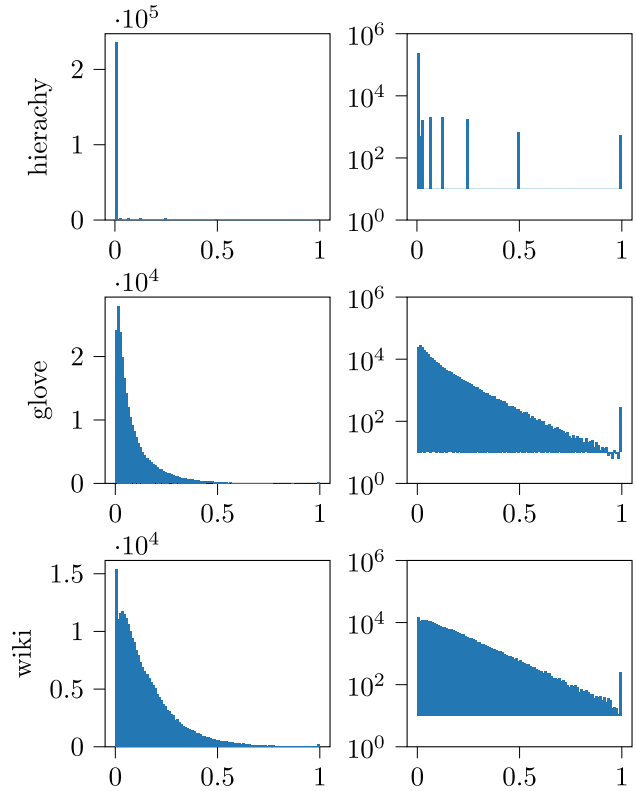


Fig. 4 Adjacency matrix distributions, processed values (normal and log scales)

compared to *glove* and *hierarchy*, the *wiki* knowledge graph yields worse results – notably for low-shot scenarios. We hypothesise that the difference in the performance of *wiki* knowledge graph is due to the low quality of embeddings, as some issues regarding their accuracy were previously reported[3]. Similarly, *hierarchy* is inherently biased since the categories sharing more common ancestors are promoted by the distance function. Therefore, the quality of *hierarchy* is questionable, especially in scenarios with a small number of classes or corner cases with classes not sharing any ancestors. In general, we hypothesise the final performance depends largely on the quality of the embeddings. In the scenario with a large number of knowledge graphs, there is also the question of how these embeddings *complete* each other. This means that KGTN with *optimal* embeddings should perform no worse than KGTN-ens with a large number of *suboptimal* embeddings. In other words, the quality of the embeddings is perhaps more important than the number of knowledge graphs.

[3] https://datascience.stackexchange.com/q/95007/8949

Table 4 Mantel test results

	KG_1	KG_1	correlation	p-value
Raw	*hierarchy*	*glove*	0.14	0.001
	hierarchy	*wiki*	0.13	0.001
	glove	*wiki*	0.16	0.001
Processed	*hierarchy*	*glove*	0.19	0.001
	hierarchy	*wiki*	0.18	0.001
	glove	*wiki*	0.44	0.001

Importance of the ensembling method Table 6 presents results for the different ensembling strategies compared to the KGTN baseline, which can be treated as a KGTN-ens model with no ensembling. Mean ensembling gave mixed results compared to the baseline (+0.34, −0.63, +0.36, −0.27 pp. for novel classes and −1.45, −1.41, +0.14, −0.17 pp. for all classes, both groups for $k \in \{1, 2, 5, 10\}$ respectively. However, using the max ensembling strategy has been better in all the cases (+0.77, +0.40, +0.29, +0.07 pp. for novel classes and +0.24, +0.18, +0.19, +0.06 pp. for all classes). A possible explanation of this effect might stem from the *winner takes all* nature of the maximum function, which chooses the most similar embedding to the given prototype and rejects other, potentially improper, embeddings. At the same time, these improper embeddings still contribute to the overall formula for the mean ensembling function. However, research on a larger number of employed knowledge graphs has to be conducted to validate this hypothesis.

Variance of the results Contrary to expectations, adding additional knowledge sources slightly increases the variance of the results in most cases (Table 7). A possible explanation of these results is the fact that KGTN-ens is not an ensembling technique in the typical sense of this word, but rather a way of choosing the embeddings of the different knowledge sources. We report results for novel classes only, as the difference in variance is amplified among these (see also Fig. 2). No significant differences in the variance of mean and max ensembling have been found. The variance of the results for baseline KGTN has been obtained using five runs of the original KGTN with *glove* embeddings.

Importance of similarity function Table 5 includes data for performing ablative studies for KGTN with the three different similarity functions: cosine similarity, inner product and Pearson correlation. [2] analysed all these for KGTN with *glove* embeddings. In general, the inner product showed the best performance. These conclusions can be extrapolated to the *wiki* graph, as the inner product usually turned out to

Table 5 Classification results for KGTN on different embeddings tested against different similarity functions

KG	sim. f.	Novel				All			
		1	2	5	10	1	2	5	10
wiki	CS	56.65	68.21	77.31	81.88	64.59	73.32	80.03	83.44
	IP	55.55	67.81	77.99	82.15	64.61	73.28	80.55	83.22
	PC	56.84	68.10	77.03	81.62	64.03	72.61	79.53	83.20
glove	CS	61.4	70.4	78.4	82.2	67.7	74.7	80.9	83.6
	IP	62.1	70.9	78.4	82.3	68.3	75.2	80.8	83.5
	PC	61.5	70.6	78.5	82.3	67.5	74.4	80.7	83.5
hierarchy	IP	60.1	69.4	78.1	82.1	67.0	74.4	80.7	83.3
(uniform)	IP	53.4	67.4	78.8	81.5	63.8	73.3	80.3	82.9
(random)	IP	54.4	67.4	77.8	81.9	64.5	73.3	80.5	83.2

Results for *glove* and *hierarchy* embeddings are provided by [2]
Bold results mean the best ones for the *wiki* graph only
Abbreviations for similarity functions: CS – cosine similarity, IP – inner product, PC – Pearson correlation

be the most effective in terms of the top-5 accuracy. Interestingly, Pearson correlation displayed the best performance for the 1-shot scenario with novel classes. Table 8 presents results for the different similarity functions used in the KGTN-ens. While the combination of *hierarchy* and *glove* embeddings was usually the best for cosine similarity as well, the results are visibly worse compared to the inner product similarity function (e.g. −3.16 pp. top-5 accuracy difference for 1-shot scenario among novel classes). Noticeably, the combination of these two graphs and cosine similarity function performed worse than KGTN solely based on *glove* embeddings (for example, there is a −2.39 pp. difference for top-5 accuracy difference for 1-shot scenario among novel classes).

Importance of feature extractor So far, we have reported ResNet-50 as a feature extractor, as this is the standard feature extractor in the few-shot learning literature. However, we also performed another set of experiments with ResNet-10 and ResNet-18. All of the results are compared to the KGTN (with *glove* KG). Similarly to ResNet-50, these models have been used and trained in the same way as KGTN [2]. This also means that SGM without generation [14] serves here as the baseline, as both KGTN and KGTN-ens are based on it. The detailed results (averaged on 5 runs) for every experiment we performed in this experiment are provided in Table 13 and 14 for ResNet-10 and ResNet-18 respectively in Appendix. We tested the KGTN-ens model with different feature extractors, similarity functions (cosine similarity, inner product, Pearson correlation coefficient), and ensembling methods (max, mean) for the same 5 runs as in the main experiment. Since that gives 96 experiments for each run, we also grouped results by the used feature extractor, k, and subset (novel/all) in order to showcase the best model in each group. For the same 5 runs as in the main experiment, we present averaged and maximum results in Tables 11 and 12 respectively.

Table 6 Knowledge graph ensembling (sum), top-5 accuracy, inner product

Type	KGs	Novel				All			
		1	2	5	10	1	2	5	10
KGTN (baseline)	g	61.96	71.08	78.53	82.48	68.34	75.27	80.92	83.40
KGTN-ens (mean)	h+g	62.30	70.45	78.90	82.21	66.89	73.86	81.06	83.22
	w+g	60.41	69.41	78.81	82.10	66.07	73.30	81.01	83.15
	w+h+g	58.74	67.95	78.74	81.70	63.90	71.43	80.91	82.88
	w+h	57.89	67.49	78.49	81.91	64.10	71.83	80.80	83.04
KGTN-ens (max)	h+g	62.73	71.48	78.83	82.56	68.58	75.45	81.12	83.46
	w+g	61.21	70.66	78.60	82.34	67.69	75.04	80.95	83.33
	w+h+g	61.32	70.77	78.70	82.38	67.85	75.06	81.06	83.35
	w+h	58.77	69.17	78.44	82.25	66.17	74.01	80.86	83.26

Abbreviations: KGs – knowledge graphs, h – *hierarchy*, g – *glove*, w – *wiki*

Table 7 Standard deviations of top-5 accuracy

Type	KGs	Novel 1	2	5	10
KGTN (baseline)	g	0.40	0.38	0.28	0.40
KGTN-ens (max)	h+g	0.53	0.33	0.32	0.30
	w+g	0.59	0.16	0.30	0.34
	w+h	0.66	0.25	0.26	0.28
	w+h+g	0.45	0.27	0.34	0.30
KGTN-ens (mean)	h+g	0.57	0.31	0.31	0.34
	w+g	0.24	0.28	0.33	0.38
	w+h	0.56	0.31	0.41	0.31
	w+h+g	0.53	0.37	0.33	0.42

KGs – knowledge graphs, Abbreviations: h – *hierarchy*, g – *glove*, w – *wiki*

Table 8 Ablation on different similarity functions used in KGTN-ens, top-5 accuracy

Type	KGs	Novel 1	2	5	10	All 1	2	5	10
KGTN	g	61.96	71.08	78.53	82.48	68.34	75.27	80.92	83.40
KGTN-ens (inner prod.)	h+g	62.73	71.48	78.83	82.56	68.58	75.45	81.12	83.46
	w+g	61.21	70.66	78.60	82.34	67.69	75.04	80.95	83.33
	w+h+g	61.32	70.77	78.70	82.38	67.85	75.06	81.06	83.35
	w+h	58.77	69.17	78.44	82.25	66.17	74.01	80.86	83.26
KGTN-ens (cosine sim.)	h+g	59.57	69.40	77.29	81.89	64.86	73.72	80.05	83.46
	w+g	58.34	68.75	77.24	81.84	64.43	73.44	80.01	83.38
	w+h+g	57.75	68.44	77.20	81.90	63.81	73.12	79.99	83.44
	w+h	57.35	68.50	77.27	81.90	63.87	73.23	80.00	83.43

Abbreviations: KGs – knowledge graphs, h – *hierarchy*, g – *glove*, w – *wiki*

Table 9 Means and standard deviations of class-wise entropy of predictions for KGTN-ens (ResNet50, inner product, max) with different combinations of embeddings (novel classes)

KGs	*k* (novel) 1	2	5	10
h+g	5.27 (±1.40)	5.27 (±1.41)	5.01 (±1.57)	5.00 (±1.57)
w+h+g	5.38 (±1.36)	5.37 (±1.37)	5.09 (±1.55)	5.08 (±1.55)

Table 10 Means and standard deviations of class-wise entropy of predictions for KGTN-ens (ResNet50, inner product, max) with different combinations of embeddings (all classes)

KGs	*k* (all) 1	2	5	10
h+g	4.50 (±1.83)	4.49 (±1.83)	4.47 (±1.82)	4.46 (±1.82)
w+h+g	4.52 (±1.83)	4.52 (±1.83)	4.49 (±1.82)	4.48 (±1.82)

For ResNet10, we can observe that the KGTN-ens model slightly outperformed the KGTN for more than half cases. Specifically, the best KGTN-ens model yielded −0.15/−0.00/0.10/−0.03 (novel) and +0.02/+0.1/+0.14/+0.21 (all) for $k = \{1, 2, 5, 10\}$ respectively (averaged on 5 different runs). While the KGTN-ens model was better slightly more often, the differences are smaller than for ResNet-50. Since the variance of the results was higher for ResNet-10, we also present the maximum results in Table 11. Taking only the best result from 5 runs, KGTN-ens yielded −0.11/+0.23/+0.16/+0.23 and +0.03/+0.11/+0.37/+0.30 over KGTN for novel/all classes respectively. Interestingly, both KGTM and KGTM-ens failed to outperform the baseline for $k = 5$ in novel classes. Among the best KGTN-ens models, *hierarchy* + *glove* was the winning knowledge graph combination most of the time. Interestingly, cosine similarity was usually the best similarity function for ResNet-10. In terms of the ensembling method, mean yielded the best results for $k = \{1, 2\}$, whereas max was the best for $k = \{5, 10\}$ (except one result on novel data). With regard to EM and SF, similar behaviour can be observed among runs with maximum results. In terms of comparing the best KGTM-ens models KGTM for ResNet-18, the mean top-5 accuracy differences were −1.48/−0.52/−0.21/+0.08 and −0.70/+0.10/−0.03/+0.25 (−0.45/+0.13/−0.20/+0.22 and −1.56/−0.37/−0.29/+0.09 for the best run). This time KGTN-ens performed better only three times. Especially the results for $k = 1$ are far beyond expectations. While results for ResNet-18 look less favourably for the method we present, we report them for the sake of scientific integrity. Interestingly, for $k = 5$ and $k = 10$, the best KGTN-ens model used *wiki* + *glove* knowledge graphs. In terms of similarity function and ensembling method, inner product and max were the best for all the cases. In general, a possible conclusion from these findings is that the effectiveness of the ensembling method and similarity function depends on the feature extractor.

Discussion on possible improvements Obtaining good class boundaries is a challenging task in few-shot learning. At the same time, the diversity of the embeddings plays a vital role in ensemble learning scenarios. This paragraph discusses possible ways of improving the performance of KGTN-ens, which are drawn mostly from ensemble learning literature. In our paper, we focused on KGTN-ens working with different knowledge graphs. However, the same knowledge graphs can be used for different KGTMs and this is a very interesting idea to explore in the future. Since the overall variance of the model is non-negligible (especially in smaller variants of ResNet), this is the case where e.g. bagging can be used. One can also consider introducing learnable weights acting both on the prototypes and the features. Given that for some weights the optimal values might be 0, this also embraces the idea of feature selection. Another idea is to randomly select a subset of embeddings for each training epoch, and then introduce e.g. Gaussian noise to the embeddings. To assess the diversity of the model (or family of the models – see below), uncertainty measures such as entropy can be used. We provide the averaged entropy per class for KGTN-ens predictions with different combinations of embeddings in Tables 9 and 10, and they reflect the top-5 accuracy. For future work, one can also calculate e.g. permutation importance to assess the influence of a particular KGTM on the final score.

5 Conclusion

In this work, we proposed KGTN-ens, which builds on KGTN and allows the incorporation of multiple knowledge sources in order to achieve better performance. We evaluated KGTN-ens on the ImageNet-FS dataset and showed that it outperforms KGTN in most of the tested settings. We also evaluated Wikidata embeddings in the same task and showed that they are not as effective as the other embeddings. We believe that the proposed approach can be used in other few-shot learning tasks and we plan to test it in the future. Although not publicly available at the time of writing this article, further work might include an evaluation of the proposed approach on the ImageNet-6K dataset [2]. A certain limitation of this study is the fact that it might not scale well for extreme classification problems, due to the calculation of pairwise distances of nodes from large knowledge graphs requiring quadratic memory complexity.

Appendix

Table 11 Best models (mean of 5 different runs) for each FE, type and *k*

FE	type	*k*	best model	EM	SF	top-5 acc
ResNet10	all	1	KGTN-ens (h+g)	mean	PC	51.97
		2	KGTN-ens (h+g)	mean	CS	61.41
		5	KGTN-ens (h+g)	max	IP	70.41
		10	KGTN-ens (w+h)	max	CS	75.78
	novel	1	KGTN (g)	-	CS	47.43
		2	KGTN (g)	-	CS	58.63
		5	baseline	-	-	69.46
		10	KGTN (g)	-	CS	74.61
ResNet18	all	1	KGTN (g)	-	IP	60.64
		2	KGTN-ens (h+g)	max	IP	68.33
		5	KGTN (g)	-	IP	76.82
		10	KGTN-ens (w+g)	max	IP	79.94
	novel	1	KGTN (g)	-	IP	55.40
		2	KGTN (g)	-	IP	64.44
		5	KGTN (g)	-	IP	74.46
		10	baseline	-	-	79.04

Table 12 Best models (max of 5 runs) for each FE, type and *k*

FE	type	*k*	best model	EM	SF	top-5 acc
ResNet10	all	1	KGTN-ens (h+g)	mean	PC	52.26
		2	KGTN-ens (h+g)	mean	CS	61.72
		5	KGTN-ens (h+g)	max	IP	70.95
		10	KGTN-ens (w+h)	max	CS	76.24
	novel	1	KGTN (g)	-	CS	48.13
		2	KGTN-ens (h+g)	mean	CS	59.22
		5	baseline	-	-	69.96
		10	KGTN-ens (h+g)	mean	CS	75.45
ResNet18	all	1	KGTN (g)	-	IP	61.09
		2	KGTN-ens (h+g)	max	IP	68.62
		5	KGTN (g)	-	IP	77.23
		10	KGTN-ens (w+h+g)	max	IP	80.38
	novel	1	KGTN (g)	-	IP	56.12
		2	KGTN (g)	-	IP	64.73
		5	KGTN (g)	-	IP	75.04
		10	baseline	-	-	79.54

Table 13 Mean top-5 accuracies averaged over 5 experiments for ResNet10_sgm

	Novel				All			
	1	2	5	10	1	2	5	10
baseline (SGM)	43.92	56.18	69.46	73.94	47.43	55.94	67.98	71.21
KGTN (g), CS	47.43	58.63	68.89	74.61	51.48	61.30	70.27	75.57
ours (h+g), CS, max	46.75	58.22	68.82	74.58	50.39	60.85	70.33	75.72
ours (w+g), CS, max	46.31	58.12	68.89	74.56	50.29	60.84	70.27	75.74
ours (w+h), CS, max	46.14	58.14	68.85	74.54	50.16	60.76	70.27	75.78
ours (w+h+g), CS, max	46.00	57.86	68.78	74.48	49.84	60.50	70.27	75.74
ours (h+g), CS, mean	47.27	58.63	68.99	74.56	51.56	61.41	70.24	75.64
ours (w+g), CS, mean	46.73	58.25	68.93	74.52	51.21	61.26	70.13	75.61
ours (w+h), CS, mean	46.52	58.24	68.90	74.48	51.11	61.13	70.19	75.66
ours (w+h+g), CS, mean	46.98	58.35	68.80	74.49	51.39	61.34	70.22	75.62
KGTN (g), IP	44.48	53.59	68.31	71.35	45.50	54.76	69.24	71.14
ours (h+g), IP, max	43.88	54.20	68.76	72.34	45.93	56.46	70.41	72.65
ours (w+g), IP, max	41.89	51.86	68.53	71.37	44.89	54.26	69.83	71.73
ours (w+h), IP, max	40.14	50.61	68.18	71.75	43.00	53.19	69.96	72.26
ours (w+h+g), IP, max	41.62	51.81	68.53	71.89	44.48	54.48	70.23	72.40
ours (h+g), IP, mean	38.21	47.93	67.31	69.73	37.90	48.22	68.20	69.65
ours (w+g), IP, mean	39.54	48.94	67.64	70.29	40.98	50.39	68.84	70.38
ours (w+h), IP, mean	35.34	45.08	66.76	69.45	35.89	45.91	67.84	69.41
ours (w+h+g), IP, mean	36.92	45.95	67.04	69.49	37.02	46.99	68.12	69.55
KGTN (g), PC	46.97	58.29	68.69	74.37	51.95	60.90	69.60	75.12
ours (h+g), PC, max	46.01	57.81	68.72	74.28	51.24	60.33	69.75	75.32
ours (w+g), PC, max	45.64	57.69	68.69	74.36	51.06	60.28	69.67	75.30
ours (w+h), PC, max	45.40	57.74	68.66	74.33	50.96	60.32	69.64	75.31
ours (w+h+g), PC, max	45.50	57.53	68.66	74.30	50.70	60.09	69.68	75.30
ours (h+g), PC, mean	46.90	58.27	68.73	74.32	51.97	60.90	69.67	75.18
ours (w+g), PC, mean	46.40	57.84	68.67	74.33	51.73	60.75	69.62	75.13
ours (w+h), PC, mean	45.90	57.76	68.69	74.23	51.39	60.64	69.57	75.19
ours (w+h+g), PC, mean	46.59	57.93	68.68	74.27	51.85	60.87	69.63	75.18

Table 14 Mean top-5 accuracies averaged over 5 experiments for ResNet18_sgm

	Novel				All			
	1	2	5	10	1	2	5	10
baseline (SGM)	47.75	61.55	73.80	79.04	53.38	64.91	75.48	79.83
KGTN (g), CS	51.60	62.47	72.54	77.88	56.19	66.29	75.07	79.68
ours (h+g), CS, max	50.79	62.24	72.58	77.96	54.79	65.87	75.12	79.81
ours (w+g), CS, max	50.39	61.99	72.58	77.96	54.83	65.81	75.11	79.83
ours (w+h), CS, max	50.23	61.92	72.54	78.01	54.67	65.75	75.07	79.84
ours (w+h+g), CS, max	49.93	61.83	72.54	77.99	54.14	65.57	75.06	79.79
ours (h+g), CS, mean	51.54	62.62	72.69	77.96	56.25	66.41	75.09	79.73
ours (w+g), CS, mean	51.00	62.22	72.56	77.98	55.84	66.23	75.01	79.70
ours (w+h), CS, mean	50.50	62.17	72.57	77.96	55.76	66.14	75.09	79.74
ours (w+h+g), CS, mean	50.97	62.40	72.54	77.94	56.10	66.31	75.12	79.76
KGTN (g), IP	55.40	64.44	74.46	78.64	60.64	68.24	76.82	79.70
ours (h+g), IP, max	53.93	63.92	74.06	78.61	59.94	68.33	76.70	79.79
ours (w+g), IP, max	53.21	63.39	74.25	78.72	59.49	67.87	76.79	79.94

Table 14 continued

	Novel				All			
	1	2	5	10	1	2	5	10
ours (w+h), IP, max	49.74	61.46	73.73	78.22	57.89	67.11	76.52	79.66
ours (w+h+g), IP, max	52.89	63.35	74.12	78.63	59.42	68.07	76.77	79.93
ours (h+g), IP, mean	51.24	61.36	74.09	78.18	56.14	65.10	76.54	79.30
ours (w+g), IP, mean	50.92	61.45	73.87	78.36	57.36	66.00	76.50	79.65
ours (w+h), IP, mean	47.45	59.16	73.71	77.98	54.29	64.09	76.18	79.20
ours (w+h+g), IP, mean	49.26	60.08	73.76	78.17	54.89	64.44	76.33	79.39
KGTN (g), PC	51.07	61.97	72.20	77.61	56.86	65.46	74.10	79.11
ours (h+g), PC, max	50.40	61.93	72.38	77.74	55.94	65.09	74.29	79.25
ours (w+g), PC, max	50.10	61.71	72.37	77.74	55.71	65.02	74.34	79.19
ours (w+h), PC, max	49.85	61.69	72.40	77.67	55.66	65.03	74.26	79.21
ours (w+h+g), PC, max	49.97	61.60	72.38	77.70	55.41	64.88	74.28	79.26
ours (h+g), PC, mean	51.30	62.23	72.27	77.68	56.84	65.65	74.13	79.18
ours (w+g), PC, mean	50.56	61.85	72.24	77.61	56.37	65.33	74.13	79.16
ours (w+h), PC, mean	49.95	61.58	72.23	77.61	56.18	65.30	74.19	79.18
ours (w+h+g), PC, mean	50.70	61.93	72.35	77.64	56.72	65.51	74.25	79.19

Acknowledgements This research was co-funded by Interreg *Österreich-Bayern 2014-2020 programme project KI-Net: Bausteine für KI-basierte Optimierungen in der industriellen Fertigung* (grant agreement: AB 292). The authors of this paper would like to thank Krzysztof Wecel for his help with conducting the experiments. We would also like to thank the anonymous reviewers for their valuable comments and suggestions, which helped to improve the paper.

Author Contributions **Dominik Filipiak**: conceptualization, data curation, formal analysis, investigation, methodology, software, validation, writing – original draft (85% of the work in total). **Anna Fensel**: conceptualization, funding acquisition, project administration, writing – review & editing (10% of the work in total). **Agata Filipowska**: writing – review & editing, resources (5% of the work in total)

Funding Open access funding provided by University of Innsbruck and Medical University of Innsbruck.

Data Availibility Statement The dataset used for this study is publicly available for research purposes in the ImageNet repository (https://www.image-net.org). The knowledge graphs, exact novel/base data splits, and the example configurations for reproducing key results obtained in this study are available on GitHub: https://github.com/DominikFilipiak/KGTN-ens

Declarations

Compliance with Ethical Standards To the best of our knowledge, this work is compliant with ethical standards. In particular, there are no potential conflicts of interest that we're aware of. This research does not involve Human Participants or Animals and therefore does not need any informed consent. The used data also does not raise any ethical issues.

Competing Interests The authors have no competing interests to declare that are relevant to the content of this article.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Bronstein MM, Bruna J, LeCun Y, Szlam A, Vandergheynst P (2017) Geometric deep learning: going beyond euclidean data. IEEE Signal Proc Mag 34:18–42
2. Chen R, Chen T, Hui X, Wu H, Li G, Lin L (2020) Knowledge graph transfer network for few-shot recognition, in: Proceedings of the AAAI Conference on Artificial Intelligence, pp. 10575–10582
3. Chen Z, Fu Y, Wang YX, Ma L, Liu W, Hebert M (2019a) Image deformation meta-networks for one–shot learning, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 8680–8689
4. Chen Z, Fu Y, Zhang Y, Jiang YG, Xue X, Sigal L (2019) Multi-level semantic feature augmentation for one-shot learning. IEEE Trans Image Process 28:4594–4605
5. Chicco D (2021) Siamese neural networks: An overview. Artificial Neural Networks 73–94
6. Cho K, Van Merriënboer B, Bahdanau D, Bengio Y (2014) On the properties of neural machine translation: Encoder-decoder approaches. arXiv preprint arXiv:1409.1259

7. Douze M, Szlam A, Hariharan B, Jégou H (2018) Low-shot learning with large-scale diffusion, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 3349–3358
8. Elsken T, Metzen JH, Hutter F (2019) Neural architecture search: A survey. J Mach Learn Res 20:1997–2017
9. Filipiak D, Fensel A, Filipowska A (2021) Mapping of imagenet and wikidata for knowledge graphs enabled computer vision, in: Business Information Systems, pp. 151–161
10. Finn C, Abbeel P, Levine S (2017) Model-agnostic meta-learning for fast adaptation of deep networks, in: International conference on machine learning, PMLR. pp. 1126–1135
11. Ge W (2018) Deep metric learning with hierarchical triplet loss, in: Proceedings of the European Conference on Computer Vision (ECCV), pp. 269–285
12. Gidaris S, Komodakis N (2018) Dynamic few-shot visual learning without forgetting, in: Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 4367–4375
13. Gilmer J, Schoenholz SS, Riley PF, Vinyals O, Dahl GE (2017) Neural message passing for quantum chemistry, in: International conference on machine learning, PMLR. pp. 1263–1272
14. Hariharan B, Girshick R (2017) Low-shot visual recognition by shrinking and hallucinating features, in: Proceedings of the IEEE International Conference on Computer Vision, pp. 3018–3027
15. He K, Zhang X, Ren S, Sun J (2016) Deep residual learning for image recognition, in: Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 770–778
16. Keriven N, Peyré G (2019) Universal invariant and equivariant graph neural networks. Advances in Neural Information Processing Systems 32
17. Krizhevsky A, Sutskever I, Hinton GE (2017) Imagenet classification with deep convolutional neural networks. Commun ACM 60:84–90
18. Lerer A, Wu L, Shen J, Lacroix T, Wehrstedt L, Bose A, Peysakhovich A (2019) Pytorch-biggraph: A large scale graph embedding system. Proceedings of Machine Learning and Systems 1:120–131
19. Li A, Luo T, Lu Z, Xiang T, Wang L (2019) Large-scale few-shot learning: Knowledge transfer with class hierarchy, in: Proceedings of the ieee/cvf conference on computer vision and pattern recognition, pp. 7212–7220
20. Li Y, Zemel R, Brockschmidt M, Tarlow D (2016) Gated graph sequence neural networks, in: Proceedings of ICLR'16
21. Mikolov T, Chen K, Corrado G, Dean J (2013) Efficient estimation of word representations in vector space. arXiv preprint arXiv:1301.3781
22. Miller GA (1995) Wordnet: a lexical database for english. Commun ACM 38:39–41
23. Monka S, Halilaj L, Rettinger A (2022) A survey on visual transfer learning using knowledge graphs. Semantic Web 1–34
24. Nielsen FÅ (2017) Wembedder: Wikidata entity embedding web service. arXiv preprint arXiv:1710.04099
25. Paszke A, Gross S, Chintala S, Chanan G, Yang E, DeVito Z, Lin Z, Desmaison A, Antiga L, Lerer A (2017) Automatic differentiation in pytorch
26. Pennington J, Socher R, Manning CD (2014) Glove: Global vectors for word representation, in: Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP), pp. 1532–1543
27. Russakovsky O, Deng J, Su H, Krause J, Satheesh S, Ma S, Huang Z, Karpathy A, Khosla A, Bernstein M, Berg AC, Fei-Fei L (2015) ImageNet Large Scale Visual Recognition Challenge. International Journal of Computer Vision (IJCV) 115:211–252. https://doi.org/10.1007/s11263-015-0816-y
28. Sanchez-Lengeling B, Reif E, Pearce A, Wiltschko AB (2021) A gentle introduction to graph neural networks. Distill https://doi.org/10.23915/distill.00033. https://distill.pub/2021/gnn-intro
29. Shen E, Brbic M, Monath N, Zhai J, Zaheer M, Leskovec J (2021) Model-agnostic graph regularization for few-shot learning. arXiv preprint arXiv:2102.07077
30. Snell J, Swersky K, Zemel R (2017) Prototypical networks for few-shot learning. Advances in neural information processing systems 30
31. Song Y, Wang T, Mondal SK, Sahoo JP (2022) A comprehensive survey of few-shot learning: Evolution, applications, challenges, and opportunities. arXiv preprint arXiv:2205.06743
32. Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser L, Polosukhin I (2017) Attention is all you need. Advances in neural information processing systems 30
33. Vinyals O, Blundell C, Lillicrap T, Wierstra D et al (2016) Matching networks for one shot learning. Advances in neural information processing systems 29
34. Vrandečić D, Krötzsch M (2014) Wikidata: a free collaborative knowledgebase. Commun ACM 57:78–85
35. Wang S, Yue J, Liu J, Tian Q, Wang M (2020) Large–scale few–shot learning via multi–modal knowledge discovery, in: European Conference on Computer Vision, Springer. pp. 718–734
36. Wang YX, Girshick R, Hebert M, Hariharan B (2018) Low-shot learning from imaginary data, in: Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 7278–7286
37. Wu Z, Pan S, Chen F, Long G, Zhang C, Philip SY (2020) A comprehensive survey on graph neural networks. IEEE Trans Neural Netw Learn Syst 32:4–24
38. Zhu Z, Xu S, Tang J, Qu M (2019) Graphvite: A high-performance cpu-gpu hybrid system for node embedding, in: The World Wide Web Conference, pp. 2494–2504

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.